

Chapter 2 Notes: The Object Model

Object-Oriented Analysis and
Design Study Notes

Contents

1	Chapter Purpose	2
1.1	Why This Chapter Matters	2
2	The Evolution of the Object Model	2
2.1	Two Historical Trends	2
2.2	Generations of Programming Languages	3
2.3	Abstraction Progression Across Generations	4
2.4	Why Earlier Approaches Became Insufficient	4
2.5	Topology of Earlier Programming Languages	4
2.5.1	First- and Early Second-Generation Topology	4
2.5.2	Late Second- and Early Third-Generation Topology	4
2.5.3	Late Third-Generation Topology	5
2.5.4	Object-Based and Object-Oriented Topology	5
2.6	Programming-in-the-Colossal	5
3	Foundations of the Object Model	6
3.1	Object Orientation as Evolution, Not Revolution	6
3.2	Sources That Shaped the Object Model	6
3.3	What Is an Object?	7
4	Object-Oriented Programming, Design, and Analysis	7
4.1	Object-Oriented Programming	7
4.1.1	Three Required Elements of OOP	7
4.2	Object-Based Versus Object-Oriented Languages	7
4.3	Object-Oriented Design	8
4.3.1	What OOD Produces	8
4.4	Object-Oriented Analysis	8
4.4.1	OOA, OOD, and OOP Relationship	8
5	Programming Paradigms and the Object Model	9
5.1	Major Programming Styles	9
5.2	No Paradigm Is Universally Best	9
6	Element 1: Abstraction	10
6.1	Definition	10
6.2	Problem Abstraction Solves	10
6.3	Why Naive Approaches Fail	10

6.4	Principles Behind Good Abstraction	10
6.4.1	Principle of Least Commitment	10
6.4.2	Principle of Least Astonishment	10
6.5	Kinds of Abstractions	11
6.6	Contracts, Clients, Servers, and Responsibilities	11
6.7	Operations, Signatures, and Protocols	11
6.8	Invariants, Preconditions, Postconditions, and Exceptions	11
6.9	Static and Dynamic Properties	12
6.10	Hydroponics Example: Temperature Sensor	13
6.11	Hydroponics Example: Active Temperature Sensor	13
6.12	Hydroponics Example: Growing Plan	13
6.13	Abstraction Design Checklist	13
6.14	Common Abstraction Mistakes	14
7	Element 2: Encapsulation	14
7.1	Definition	14
7.2	Encapsulation Versus Information Hiding	14
7.3	Problem Encapsulation Solves	15
7.4	Interface and Implementation	15
7.5	Hydroponics Example: Heater	15
7.6	Hydroponics Example: Growing Plan Representation	16
7.7	Encapsulation and Change	16
7.8	Limitations of Encapsulation	17
7.9	Common Encapsulation Mistakes	17
8	Element 3: Modularity	17
8.1	Definition	17
8.2	Problem Modularity Solves	17
8.3	Modules as Physical Containers	18
8.4	Coupling and Cohesion	18
8.5	Why Arbitrary Modularization Fails	18
8.6	Technical Factors in Modularization	18
8.7	Nontechnical Factors in Modularization	19
8.8	Logical Design Versus Physical Design	19
8.9	Hydroponics Example: Modules	19
8.10	Object-Oriented Systems Have No Single Top	20
9	Element 4: Hierarchy	20
9.1	Definition	20
9.2	Why Hierarchy Matters	20
9.3	Class Hierarchy: Inheritance	20
9.4	Generalization and Specialization	20
9.5	Hydroponics Example: Growing Plans	20
9.6	Benefits of Inheritance	21
9.7	Inheritance and Encapsulation Tension	21
9.8	Multiple Inheritance	21
9.8.1	Mixin Classes	21
9.9	Problems With Multiple Inheritance	21

9.10	The Diamond Problem	22
9.11	Object Hierarchy: Aggregation and Composition	22
9.12	Aggregation Versus Composition	22
9.13	Levels of Abstraction	23
9.14	Common Hierarchy Mistakes	23
10	Element 5: Typing	23
10.1	Definition	23
10.2	Type and Class	23
10.3	Problem Typing Solves	23
10.4	Strong, Weak, and Untyped Languages	23
10.5	Benefits of Strong Typing	24
10.6	Costs of Strong Typing	24
10.7	Static Versus Dynamic Typing	24
10.8	Polymorphism	24
10.9	Runtime Type Identification	25
11	Element 6: Concurrency	25
11.1	Definition	25
11.2	Why Concurrency Matters	25
11.3	Thread of Control	25
11.4	True Versus Simulated Concurrency	26
11.5	Heavyweight Versus Lightweight Processes	26
11.6	Concurrency Hazards	26
11.7	Object Model View of Concurrency	26
11.8	Hydroponics Example: Active Sensor	27
11.9	Concurrency and Encapsulation	27
12	Element 7: Persistence	28
12.1	Definition	28
12.2	Continuum of Object Existence	28
12.3	Problem Persistence Solves	28
12.4	Object-Relational Tension	28
12.5	Approaches to Persistence	29
12.6	Persistence of Class and State	29
13	Applying the Object Model	29
13.1	Five Benefits of Applying the Object Model	29
13.2	How the Object Model Supports Complex-System Attributes	30
13.3	Open Issues Introduced by the Chapter	30
14	UML and Architecture Implications	30
14.1	What UML Diagrams Communicate	30
14.2	When to Use UML for Chapter 2 Concepts	30
14.3	How to Interpret OO Diagrams	31
14.4	Common Diagram Mistakes	31
15	Design Tradeoffs Across Object Model Elements	31

16 Key Takeaways	32
17 Common Pitfalls	33
17.1 Pitfall 1: Treating Classes as Records	33
17.2 Pitfall 2: Confusing Abstraction With Vagueness	33
17.3 Pitfall 3: Exposing Too Much Interface	33
17.4 Pitfall 4: Using Inheritance for Code Reuse Alone	33
17.5 Pitfall 5: Creating Arbitrary Modules	33
17.6 Pitfall 6: Ignoring Object Lifetimes	33
17.7 Pitfall 7: Adding Concurrency Without Redesigning Semantics	33
17.8 Pitfall 8: Overusing Runtime Type Checks	34
18 Interview Questions	34
18.1 Question 1	34
18.2 Question 2	34
18.3 Question 3	34
18.4 Question 4	34
18.5 Question 5	34
18.6 Question 6	34
18.7 Question 7	35
18.8 Question 8	35
18.9 Question 9	35
18.10 Question 10	35
18.11 Question 11	35
18.12 Question 12	35
18.13 Question 13	36
18.14 Question 14	36
18.15 Question 15	36
18.16 Question 16	36
18.17 Question 17	36
18.18 Question 18	36
19 Engineering Insights	36
19.1 Insight 1: Object-Oriented Design Is Responsibility Assignment	36
19.2 Insight 2: Interfaces Are Commitments	37
19.3 Insight 3: Reuse Requires Upfront Design Investment	37
19.4 Insight 4: Inheritance Is a Semantic Tool, Not Just a Coding Tool	37
19.5 Insight 5: Persistence and Concurrency Change Object Semantics	37
19.6 Insight 6: Modules Are Management Boundaries	37
20 Connections to Other Topics	37
20.1 Connection to Chapter 1: Complexity	37
20.2 Connection to SOLID Principles	37
20.3 Connection to Design Patterns	38
20.4 Connection to Software Architecture	38
20.5 Connection to Testing	38

21 Practical Applications	38
21.1 Application 1: Designing a Domain Model	38
21.2 Application 2: Refactoring Procedural Code	38
21.3 Application 3: Designing Package Boundaries	39
21.4 Application 4: Choosing Inheritance or Composition	39
21.5 Application 5: Designing Persistent Domain Objects	39
21.6 Application 6: Designing Concurrent Objects	39
22 Summary Cheat Sheet	40
22.1 Object Model Elements	40
22.2 Most Important Comparisons	40
22.3 Fast Exam Review	41
22.4 One-Page Mental Model	41

1 Chapter Purpose

Chapter 2 explains the **object model**: the conceptual foundation behind object-oriented analysis, design, and programming. The object model is not merely a syntax feature of languages such as Java or C++. It is a way to decompose complex software systems around **objects, classes, responsibilities, collaborations, and hierarchies** rather than around global data and procedures.

Core Idea

The object model combines seven principles:

1. **Abstraction**
2. **Encapsulation**
3. **Modularity**
4. **Hierarchy**
5. **Typing**
6. **Concurrency**
7. **Persistence**

The first four are the major structural principles. Typing, concurrency, and persistence add important semantic and runtime support.

1.1 Why This Chapter Matters

Software systems become difficult because they contain many interacting concepts, many changing requirements, and many implementation details. The object model matters because it gives engineers a disciplined way to:

- isolate complexity behind stable boundaries;
- model real-world and solution-domain entities directly;
- localize change;
- improve reuse through classes, inheritance, and frameworks;
- reason about both static structure and dynamic behavior;
- design systems that can evolve rather than collapse under maintenance.

2 The Evolution of the Object Model

2.1 Two Historical Trends

The chapter identifies two broad trends in software engineering history:

1. **A shift from programming-in-the-small to programming-in-the-large.**
2. **The evolution of higher-order programming languages.**

Programming-in-the-small focuses on writing individual algorithms or small programs. **Programming-in-the-large** focuses on engineering systems that are too large for one person to fully understand, requiring decomposition, abstraction, modularity, testing discipline, team coordination, and long-term maintainability.

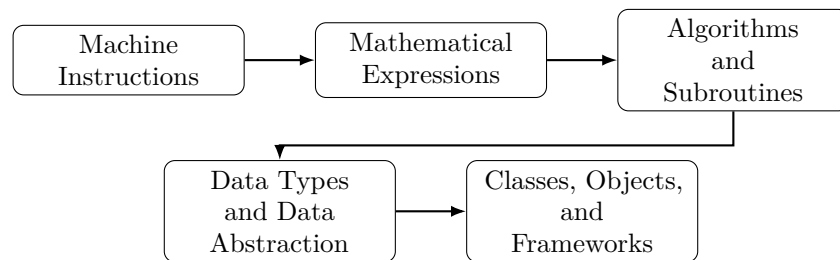
Engineering Interpretation

Language evolution mirrors the search for better abstraction mechanisms. Each generation moved software farther away from raw machine details and closer to the concepts of the problem domain.

2.2 Generations of Programming Languages

Generation Era	Representative Examples	Main Abstraction Contribution
First generation high-level languages	FORTRAN I, ALGOL 58, Flowmatic, IPL V	Mathematical expression abstraction. Programmers could write formulas closer to engineering and scientific notation instead of machine instructions.
Second generation	FORTRAN II, ALGOL 60, COBOL, Lisp	Algorithmic and procedural abstraction. Subroutines, block structure, data types, file handling, list processing, pointers, and garbage collection emerged.
Third generation	PL/I, ALGOL 68, Pascal, Simula	Data abstraction became more explicit. Languages began to describe and enforce relationships among data values through types. Simula introduced classes and data abstraction.
Generation gap	C, FORTRAN 77, many experimental languages	Many languages were created; few survived. Important mechanisms from experimental languages influenced later mainstream languages. C emphasized efficiency and small executables.
Object-orientation boom	Smalltalk 80, C++, Ada83, Eiffel	Objects and classes became central abstractions. Smalltalk emphasized pure object orientation; C++ combined C efficiency with Simula-like classes.
Framework era	Java, Python, J2EE, .NET, C#, Visual Basic .NET	Frameworks supplied reusable components, services, libraries, and runtime environments, making component reuse and enterprise development more practical.

2.3 Abstraction Progression Across Generations



2.4 Why Earlier Approaches Became Insufficient

Early programming languages worked well when programs were small. As software became larger:

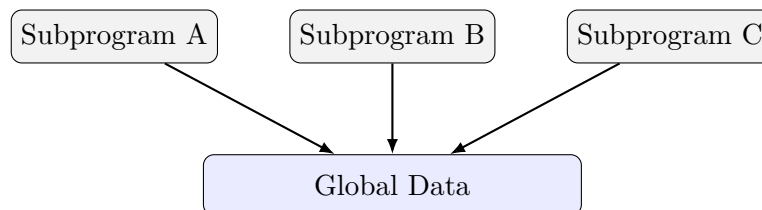
- global data caused ripple effects;
- procedure call graphs became tangled;
- design intent was not enforced by the language;
- data meanings were implicit rather than protected;
- module interfaces were often weakly checked;
- maintenance introduced entropy: cross-coupling, hidden assumptions, and unclear control flow.

2.5 Topology of Earlier Programming Languages

2.5.1 First- and Early Second-Generation Topology

Early programs were often organized around:

- **global data**; and
- **subprograms** that operated on that data.



Pitfall

Global data makes local reasoning almost impossible. Any subprogram can depend on or corrupt shared state, so a small change can have system-wide effects.

2.5.2 Late Second- and Early Third-Generation Topology

Subprograms became recognized as **procedural abstractions**. This led to:

- parameter passing mechanisms;
- nested subprograms;
- scope and visibility rules;

- structured programming;
- structured design methods.

This improved algorithmic organization but still did not fully solve data design and programming-in-the-large.

2.5.3 Late Third-Generation Topology

Separately compiled modules became important because large teams needed to develop different parts of a system independently. However, early modules were often arbitrary containers for data and subprograms, not true semantic abstractions.

Why This Matters

A module boundary helps only when it captures a meaningful design decision. If a module is just a random bag of functions, it may reduce compile-time complexity but not conceptual complexity.

2.5.4 Object-Based and Object-Oriented Topology

Object-oriented languages shift the primary building blocks from algorithms to:

- **classes**;
- **objects**;
- **modules containing logically related classes and objects**;
- **collaborations among objects**.

The physical structure of a small-to-moderate object-oriented application often resembles a graph rather than a tree. Objects collaborate across module boundaries, and global data is minimized.

Nouns Versus Verbs

Procedure-oriented design organizes software around **verbs**: functions, algorithms, and steps. Object-oriented design organizes software around **nouns**: domain entities, objects, classes, and responsibilities.

2.6 Programming-in-the-Colossal

For very large systems, even individual classes and modules are not enough. Large systems require **clusters of abstractions** layered on top of each other:

- low-level utility objects;
- domain objects;
- service objects;
- subsystem-level abstractions;
- application-level architecture;
- distributed components and frameworks.

3 Foundations of the Object Model

3.1 Object Orientation as Evolution, Not Revolution

Object-oriented analysis and design did not discard earlier software engineering ideas. It built on:

- procedural abstraction;
- structured programming;
- modularization;
- information hiding;
- abstract data types;
- database modeling;
- AI knowledge representation;
- cognitive views of how humans classify the world.

Important Insight

Object orientation is not valuable because it is newer than structured design. It is valuable because it provides a more scalable decomposition strategy for systems whose complexity is driven by interacting entities, state, behavior, and change.

3.2 Sources That Shaped the Object Model

Source		Contribution to the Object Model
Computer architecture		Capability-based and descriptor-based architectures influenced the idea of protected objects with controlled access.
Operating systems		Layered state machines and object-oriented operating systems influenced active entities, protection, and resource management.
Programming languages	lan-	Simula introduced classes and objects; Smalltalk pushed the paradigm further by treating everything as an object; C++, Java, Eiffel, Ada, and others adapted object concepts to industrial programming.
Programming methodology		Dijkstra emphasized layers of abstraction; Parnas emphasized information hiding; Liskov, Guttag, Shaw, and others developed abstract data type theory.
Database models		Entity-relationship modeling encouraged viewing systems in terms of entities, attributes, and relationships.
Artificial intelligence		Frame-based knowledge representation modeled real-world objects using slots, relationships, and behaviors.
Philosophy and cognitive science		Humans naturally classify the world into objects, categories, relationships, and behaviors; this makes object modeling intuitive.

3.3 What Is an Object?

An object is a tangible or conceptual entity that:

- has identity;
- maintains state;
- exposes behavior;
- preserves invariants;
- collaborates with other objects;
- belongs to some class or type.

Object Integrity

An object should only change state, behave, or relate to other objects in ways appropriate to its abstraction. For example, an elevator object should preserve the invariant that it travels only within its shaft. Violating such invariants means the model no longer represents the real concept correctly.

4 Object-Oriented Programming, Design, and Analysis

4.1 Object-Oriented Programming

Object-oriented programming (OOP) is a method of implementation in which programs are organized as cooperative collections of objects. Each object is an instance of a class, and classes are organized into inheritance relationships.

4.1.1 Three Required Elements of OOP

A program is object-oriented only when all three of the following are present:

1. **Objects are the fundamental logical building blocks.** The system is not primarily a tree of algorithms; it is a collaboration of objects.
2. **Every object is an instance of a class.** Objects share common structure and behavior through class definitions.
3. **Classes participate in inheritance relationships.** The system can express “is-a” relationships among abstractions.

Common Mistake

Using classes without inheritance or polymorphism is often just programming with abstract data types. It may be useful, but it is not fully object-oriented in the strict sense used by the chapter.

4.2 Object-Based Versus Object-Oriented Languages

Criterion	Object-Based	Object-Oriented
Objects with hidden state	Yes	Yes
Named operations / methods	Yes	Yes
Associated type or class	Yes	Yes
Inheritance among types/classes	Not required	Required
Typical use	Encapsulated data abstraction	Full OO design using generalization, specialization, and polymorphism
Strict classification example	Ada83 as object-based	Smalltalk, C++, Eiffel, Java, C#, Object Pascal as object-oriented

4.3 Object-Oriented Design

Object-oriented design (OOD) is a method of design that includes:

- object-oriented decomposition; and
- notation for depicting logical, physical, static, and dynamic models of the system.

4.3.1 What OOD Produces

OOD produces models such as:

- **logical static models:** class structures, object structures, inheritance, associations;
- **logical dynamic models:** object interactions, responsibilities, state changes;
- **physical static models:** modules, packages, components, deployment units;
- **physical dynamic models:** processes, threads, tasks, communication paths.

OOD Versus Structured Design

Structured design decomposes systems primarily by algorithms and data flow. Object-oriented design decomposes systems by classes, objects, responsibilities, and collaborations.

4.4 Object-Oriented Analysis

Object-oriented analysis (OOA) examines requirements from the perspective of the classes and objects found in the vocabulary of the problem domain.

4.4.1 OOA, OOD, and OOP Relationship



Engineering Interpretation

OOA asks: “What domain concepts exist?”

OOD asks: “How should these concepts collaborate and be structured?”

OOP asks: “How do we implement the design using language mechanisms?”

5 Programming Paradigms and the Object Model

5.1 Major Programming Styles

Programming Style	Primary Abstraction	Best Fit
Procedure-oriented	Algorithms	Computation-heavy tasks, numerical routines, transformations with clear step-by-step logic.
Object-oriented	Classes and objects	Systems involving rich domain entities, state, identity, behavior, change, and collaboration.
Logic-oriented	Goals expressed in predicate calculus	Constraint solving, symbolic reasoning, theorem proving.
Rule-oriented	If-then rules	Expert systems and knowledge bases.
Constraint-oriented	Invariant relationships	Layout systems, scheduling, optimization, and systems where relationships must always hold.

5.2 No Paradigm Is Universally Best

A strong design may combine paradigms:

- object-oriented architecture for high-level structure;
- procedural algorithms inside performance-critical methods;
- rule engines for business rules;
- constraints for validation or scheduling;
- logic-based approaches for inference.

Practical Insight

Object orientation often works best as the architectural framework that organizes other paradigms. For example, a **RulesEngine** object may internally use rule-oriented logic while still participating in an object-oriented system.

6 Element 1: Abstraction

6.1 Definition

Abstraction denotes the essential characteristics of an object that distinguish it from all other kinds of objects and provide clear conceptual boundaries from the viewer's perspective.

Core Meaning

Abstraction focuses on the **outside view** of an object: what the object is, what responsibilities it has, and how clients may use it. It intentionally suppresses irrelevant implementation details.

6.2 Problem Abstraction Solves

Without abstraction, every design discussion collapses into low-level details:

- memory representation;
- algorithms;
- I/O protocols;
- database schemas;
- message formats;
- hardware communication.

Abstraction lets engineers reason about one concept at a time.

6.3 Why Naive Approaches Fail

A naive approach exposes too much detail too early:

- Clients depend on implementation choices.
- Changing representation breaks unrelated code.
- Domain concepts are buried under technical details.
- Requirements discussions become confused with implementation decisions.

6.4 Principles Behind Good Abstraction

6.4.1 Principle of Least Commitment

Expose only the behavior that clients truly need. Avoid committing prematurely to implementation details.

6.4.2 Principle of Least Astonishment

An abstraction should behave as clients reasonably expect. It should not contain surprising side effects or hidden responsibilities outside its conceptual boundary.

Example

A `TemperatureSensor` should report temperature and perhaps calibrate itself. It should not silently change greenhouse lighting, update billing records, or restart the network connection.

6.5 Kinds of Abstractions

Abstraction Type	Meaning	Quality
Entity abstraction	Represents a useful problem-domain or solution-domain entity.	Most desirable.
Action abstraction	Provides a generalized set of operations that perform a related function.	Useful when behavior is central.
Virtual machine abstraction	Groups operations used by a higher-level control layer or implemented using lower-level operations.	Useful for layered systems.
Coincidental abstraction	Packages unrelated operations together.	Usually poor design.

6.6 Contracts, Clients, Servers, and Responsibilities

A **client** is an object that uses another object. A **server** is an object that provides services to clients.

The outside view of an object forms a **contract**. This contract states:

- what services the object provides;
- what assumptions clients may rely on;
- what inputs are legal;
- what outputs or state changes are promised;
- what orderings of operations are valid.

6.7 Operations, Signatures, and Protocols

Term	Meaning
Operation / method / member function	A behavior clients may request from an object. These terms come from different language cultures but are conceptually similar.
Signature	The formal arguments and return type of an operation.
Protocol	The full set of operations clients may invoke, plus the legal orderings in which those operations may be invoked.
Responsibility	Behavior for which an object is accountable.

6.8 Invariants, Preconditions, Postconditions, and Exceptions

An **invariant** is a Boolean condition that must remain true for an abstraction to stay valid.

Concept	Meaning
Invariant	A condition that must be preserved. Example: a temperature reading must remain within the sensor's measurable range.
Precondition	A condition the client must satisfy before calling an operation.
Postcondition	A condition the server must satisfy after completing an operation.
Exception	A signal that an invariant, precondition, or postcondition cannot be satisfied.

Listing 1: Contract-style reasoning for a temperature sensor.

```
final class TemperatureSensor {
    private final Location location;
    private double currentTemperature;

    '''
    // Invariant:
    // - location is never null.
    // - currentTemperature is within the measurable range.

    public TemperatureSensor(Location location) {
        if (location == null) {
            throw new IllegalArgumentException("location is required");
        }
        this.location = location;
    }

    // Postcondition: returns the latest known temperature.
    public double readTemperature() {
        return currentTemperature;
    }

    // Precondition: reference temperature must be within calibration range.
    public void calibrate(double referenceTemperature) {
        if (referenceTemperature < -50 || referenceTemperature > 150) {
            throw new IllegalArgumentException("reference out of range");
        }
        // Implementation detail hidden from clients.
    }
    '''
}
```

6.9 Static and Dynamic Properties

Every abstraction has:

- **static properties:** structural attributes such as name, identity, location, supported operations;
- **dynamic properties:** values and behavior that change over the object's lifetime.

6.10 Hydroponics Example: Temperature Sensor

A passive `TemperatureSensor` abstraction has:

- **important characteristics:** temperature, location;
- **responsibilities:** report current temperature, calibrate.

TemperatureSensor	
Characteristics:	temperature location
Responsibilities:	report current temperature calibrate

6.11 Hydroponics Example: Active Temperature Sensor

An active sensor changes responsibility. Instead of only reporting when asked, it reports when temperature changes enough from a setpoint.

ActiveTemperatureSensor		
Characteristics:	temperature loca- tion setpoint	
Responsibilities:	report current tem- perature calibrate establish set- point notify on threshold crossing	

6.12 Hydroponics Example: Growing Plan

A `GrowingPlan` describes how conditions such as temperature, light, pH, and nutrients should change over time for a crop.

- It is an entity abstraction because it belongs to the vocabulary of the domain.
- It should know the plan, not necessarily execute the plan.
- Execution may belong to a separate `PlanController`.

Separation of Responsibilities

A `GrowingPlan` stores and explains what should happen. A `PlanController` performs or coordinates those actions. Mixing these responsibilities would make the abstraction less cohesive.

6.13 Abstraction Design Checklist

A good abstraction should answer:

1. What concept does this object represent?
2. What responsibilities does it own?
3. What operations are clients allowed to invoke?
4. What invariants must always hold?
5. What details should clients not know?

6. What collaborators does it need?
7. Is the abstraction named in the language of the problem domain or solution domain?

6.14 Common Abstraction Mistakes

- Creating coincidental abstractions that group unrelated methods.
- Exposing implementation details in the public interface.
- Giving one object too many unrelated responsibilities.
- Confusing a data record with a true object.
- Naming classes after technical mechanisms rather than domain concepts when domain concepts are available.
- Designing only passive data holders and putting all behavior in service objects.

7 Element 2: Encapsulation

7.1 Definition

Encapsulation is the process of compartmentalizing the elements of an abstraction that constitute its structure and behavior. It separates the contractual interface of an abstraction from its implementation.

Core Meaning

Abstraction defines the outside view. Encapsulation protects the inside view.

7.2 Encapsulation Versus Information Hiding

Encapsulation is commonly achieved through **information hiding**. Information hiding means hiding all secrets that do not contribute to the object's essential behavior.

Concept	Meaning
Data hiding	Hiding internal state fields.
Information hiding	Hiding data representation, algorithms, communication mechanisms, caching decisions, optimization strategies, and any implementation detail likely to change.
Encapsulation	Packaging structure and behavior so the interface remains separate from the implementation.

Important Distinction

Encapsulation is stronger than merely making fields private. A class can have private fields and still have poor encapsulation if its public methods expose internal representation or force clients to understand internal rules.

7.3 Problem Encapsulation Solves

Encapsulation solves the problem of **uncontrolled dependency**. Without encapsulation:

- clients depend on internal representation;
- implementation changes break many clients;
- optimizations become dangerous;
- invariants are harder to protect;
- responsibilities become scattered.

7.4 Interface and Implementation

Every well-encapsulated class has two conceptual parts:

Part	Purpose
Interface	Captures the outside view: operations, responsibilities, promises, and assumptions clients may rely on.
Implementation	Captures the inside view: representation, helper methods, algorithms, protocols, resource management, and hidden mechanisms.

7.5 Hydroponics Example: Heater

A **Heater** may expose only three operations:

- turn on;
- turn off;
- provide status.

Its implementation may use serial lines, memory-mapped I/O, relays, network messages, or hardware drivers. Clients should not care.

Heater		
Characteristics:	location	status
Responsibilities:	turn on	turn off
	provide status	

Listing 2: Encapsulation shields clients from hardware implementation choices.

```
interface Heater {
    void turnOn();
    void turnOff();
    boolean isRunning();
}

final class SerialLineHeater implements Heater {
    private final SerialPort port;
    private final int heaterId;
    ...
}
```

```

    public void turnOn() {
        port.send("HEATER " + heaterId + " ON");
    }

    public void turnOff() {
        port.send("HEATER " + heaterId + " OFF");
    }

    public boolean isRunning() {
        return port.query("HEATER " + heaterId + " STATUS").equals("ON");
    }
    '''

}

final class MemoryMappedHeater implements Heater {
    private final HardwareRegister register;

    '''
    public void turnOn() {
        register.setBit(1);
    }

    public void turnOff() {
        register.clearBit(1);
    }

    public boolean isRunning() {
        return register.isBitSet(1);
    }
    '''

}

```

The client depends on `Heater`, not on `SerialLineHeater` or `MemoryMappedHeater`. This localizes change.

7.6 Hydroponics Example: Growing Plan Representation

A `GrowingPlan` could internally store time/action pairs in a dictionary or hash table. It might store only changes rather than storing every hour of the season. The extrapolation strategy is an implementation secret.

Engineering Insight

A good encapsulation hides decisions that are likely to change: data structures, compression strategies, caching, extrapolation, communication protocols, and optimization techniques.

7.7 Encapsulation and Change

Encapsulation enables change by making the public contract stable while allowing implementation to evolve. For example:

- replace a list with a hash map;
- cache expensive calculations;
- migrate from local storage to a remote API;
- change a communication protocol;
- optimize memory representation;
- add synchronization internally.

7.8 Limitations of Encapsulation

Encapsulation prevents accidental misuse, not malicious misuse. A language can make implementation details harder to access, but it cannot prevent a developer from reading source code or deliberately violating design intent where access is available.

7.9 Common Encapsulation Mistakes

- Exposing mutable internal collections directly.
- Using public fields instead of methods that preserve invariants.
- Adding getters and setters for every field without considering the abstraction.
- Letting clients perform multi-step state changes that should be atomic.
- Leaking storage, protocol, or database details into domain interfaces.
- Confusing encapsulation with secrecy rather than dependency control.

8 Element 3: Modularity

8.1 Definition

Modularity is the property of a system decomposed into a set of cohesive and loosely coupled modules.

Core Meaning

Classes and objects form the **logical structure** of a system. Modules, packages, libraries, and components form the **physical architecture** that packages those abstractions.

8.2 Problem Modularity Solves

Modularity manages programming-in-the-large by:

- creating documented boundaries;
- supporting separate compilation;
- enabling parallel team development;
- localizing change;
- supporting reuse;
- reducing cognitive load;
- organizing documentation and configuration management.

8.3 Modules as Physical Containers

Different languages provide different module mechanisms:

- Java uses packages and modules;
- C++ uses headers, translation units, namespaces, and modules in modern C++;
- Ada and Object Pascal provide module-like constructs;
- Smalltalk traditionally emphasizes classes more than modules.

8.4 Coupling and Cohesion

Property	Meaning	Goal
Cohesion	Degree to which elements inside a module belong together.	High cohesion.
Coupling	Degree to which one module depends on another.	Low coupling.
Interface width	Amount of information exposed to other modules.	As narrow as practical.
Secret	Design decision hidden inside a module.	Hide likely changes.

Rule of Thumb

Put logically related abstractions together. Expose only what other modules must use. Hide everything else in the implementation.

8.5 Why Arbitrary Modularization Fails

A naive design might put every message class or every tiny class in its own module. This may appear modular but can cause:

- too many files and interfaces;
- documentation overhead;
- poor discoverability;
- excessive recompilation;
- fragmented design decisions;
- unnecessary dependency management.

Arbitrary modularization can be worse than no modularization because it creates boundaries without meaning.

8.6 Technical Factors in Modularization

Good module boundaries consider:

- likely future changes;
- compilation dependencies;

- reuse boundaries;
- module size limits;
- locality of reference and runtime performance;
- visibility of declarations;
- stability of interfaces;
- dependency direction.

8.7 Nontechnical Factors in Modularization

Real projects also choose modules based on:

- team ownership;
- subcontractor boundaries;
- documentation units;
- configuration management;
- security classification;
- release planning;
- organizational communication paths.

Architecture Implication

Module boundaries are often socio-technical boundaries. They reflect both design logic and how humans coordinate work.

8.8 Logical Design Versus Physical Design

Design Concern	Primary Question	Artifacts
Logical design	What are the important abstractions and collaborations?	Classes, objects, responsibilities, relationships.
Physical design	How are abstractions packaged, compiled, deployed, and owned?	Modules, packages, components, libraries, services.

These decisions are iterative. Engineers cannot complete all logical design first and then all physical design; each influences the other.

8.9 Hydroponics Example: Modules

A hydroponics system might include modules such as:

- **growingplan:** `GrowingPlan`, `FruitGrowingPlan`, `VegetableGrowingPlan`;
- **sensors:** temperature, humidity, pH, nutrient sensors;
- **actuators:** heaters, lights, pumps, nutrient injectors;
- **ui:** workstation screens and user interaction;
- **control:** controllers coordinating plans, sensors, and actuators.

8.10 Object-Oriented Systems Have No Single Top

In traditional structured design, the main program often appears to be the root of the system. In object-oriented design, the system is better viewed as offering a set of services through collaborating objects. The `main` function may simply assemble objects and start execution.

9 Element 4: Hierarchy

9.1 Definition

Hierarchy is a ranking or ordering of abstractions.

The two most important hierarchies in object-oriented systems are:

1. **Class hierarchy:** ‘is-a’ relationships, usually implemented with inheritance.
2. **Object hierarchy:** ‘part-of’ relationships, usually implemented with aggregation or composition.

9.2 Why Hierarchy Matters

Even after abstraction, encapsulation, and modularity, a system may contain more abstractions than humans can understand at once. Hierarchy reduces cognitive load by organizing abstractions into levels.

9.3 Class Hierarchy: Inheritance

Inheritance defines a relationship among classes in which a subclass shares, extends, or redefines the structure or behavior of one or more superclasses.

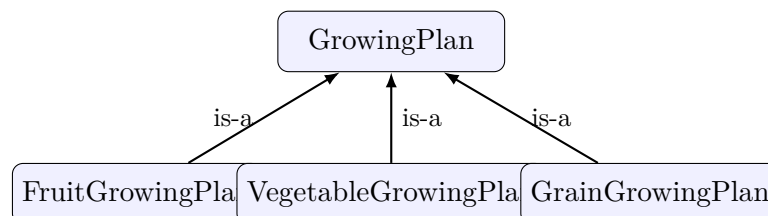
Inheritance Litmus Test

If B is not truly a kind of A, then B should not inherit from A.

9.4 Generalization and Specialization

- A superclass is more general.
- A subclass is more specialized.
- Shared structure and behavior should migrate upward to common superclasses.
- Specialized behavior belongs in subclasses.

9.5 Hydroponics Example: Growing Plans



9.6 Benefits of Inheritance

- Expresses generalization/specialization relationships.
- Reduces duplication.
- Supports polymorphism.
- Improves conceptual economy.
- Helps introduce new concepts by relating them to known concepts.

9.7 Inheritance and Encapsulation Tension

Inheritance can weaken encapsulation because subclasses may need access to superclass internals. Common access levels reflect different tradeoffs:

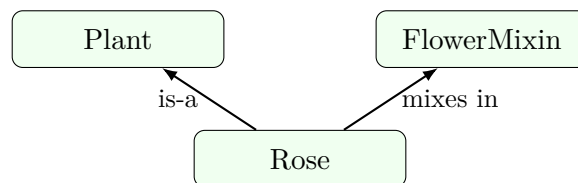
Access Level	Meaning	Risk
Private	Accessible only inside the class itself.	Strong encapsulation, less subclass flexibility.
Protected	Accessible inside the class and subclasses.	More flexible, but can expose internals to subclasses.
Public	Accessible to all clients.	Necessary for interface, dangerous if too broad.

9.8 Multiple Inheritance

Multiple inheritance allows a class to inherit from more than one superclass. It is useful when an abstraction truly combines multiple independent roles.

9.8.1 Mixin Classes

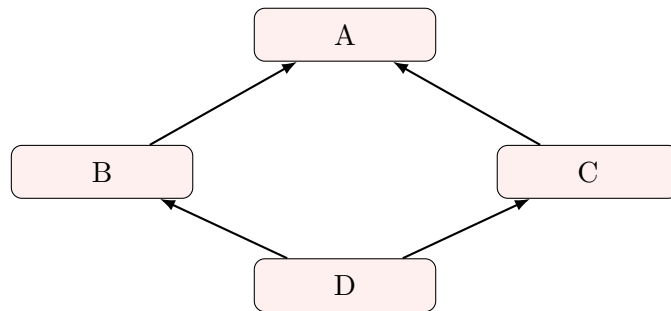
A **mixin** class captures a reusable set of properties or behavior intended to be combined with other classes.



9.9 Problems With Multiple Inheritance

Problem	Meaning	Design Response
Name clash	Two superclasses define a field or operation with the same name or signature.	Require qualification, rename, refactor roles, or prefer composition.
Repeated inheritance	Two peer superclasses share a common superclass, creating a diamond.	Use virtual base classes, interfaces, traits, or composition depending on language.
Conceptual misuse	A class inherits from another class even though it is not a true subtype.	Apply the “is-a” test and replace false inheritance with aggregation.

9.10 The Diamond Problem



The key question: does D contain one shared copy of A’s state or two separate copies? Different languages answer differently.

9.11 Object Hierarchy: Aggregation and Composition

Aggregation describes “part-of” relationships. For example:

- plants are part of a garden;
- a growing plan is part of a garden;
- sensors and actuators are part of a greenhouse control system.

9.12 Aggregation Versus Composition

Relationship	Meaning	Lifetime Implication
Aggregation	Weak whole-part relationship. Parts may exist independently.	Removing the whole does not necessarily destroy the parts.
Composition	Strong whole-part ownership. Parts are intrinsic to the whole.	Destroying the whole usually destroys the parts.

Hydroponics Example

A **Garden** may aggregate **Plant** objects because plants can be transplanted. A **Garden** may compose a **GrowingPlan** if the plan exists only as part of that garden.

9.13 Levels of Abstraction

In an **is-a** hierarchy, higher-level abstractions are more general. In a **part-of** hierarchy, the whole is at a higher level of abstraction than its parts.

9.14 Common Hierarchy Mistakes

- Using inheritance for code reuse when there is no true subtype relationship.
- Building inheritance trees too early before domain concepts are stable.
- Making superclass internals protected by default.
- Creating deep hierarchies that are hard to understand.
- Ignoring composition when it would be simpler and more flexible.
- Overusing multiple inheritance or mixins without clear semantic roles.

10 Element 5: Typing

10.1 Definition

Typing is the enforcement of the class of an object, such that objects of different types may not be interchanged, or may be interchanged only in restricted ways.

Core Meaning

Typing lets the programming language enforce design decisions.

10.2 Type and Class

For practical design purposes, a class can be understood as implementing a type. Some languages distinguish type and class more carefully, but for most OO design work they are closely related.

10.3 Problem Typing Solves

Typing prevents accidental mixing of incompatible abstractions:

- distance divided by time produces speed, not weight;
- a `Heater` should not be passed where a `TemperatureSensor` is required;
- a `GrowingPlan` should not be treated as a `Plant`.

10.4 Strong, Weak, and Untyped Languages

Typing Style	Meaning	Tradeoff
Strong typing	Type conformance is strictly enforced. Operations cannot be called unless supported by the object's class or superclass.	Safer, more explicit, sometimes less flexible.
Weak typing	Type rules are less strict and may allow implicit conversions or unsafe operations.	Flexible, but errors can appear later.

Typing Style	Meaning	Tradeoff
Untyped or dynamically flexible style	Variables may hold many kinds of objects, with checks delayed until runtime.	Very flexible, but requires discipline and testing.

10.5 Benefits of Strong Typing

- catches errors earlier;
- improves documentation;
- helps compilers optimize;
- reduces mysterious runtime crashes;
- supports large-team development by making contracts explicit.

10.6 Costs of Strong Typing

- can introduce semantic dependencies;
- interface changes may force recompilation or ripple through subclasses;
- may require extra generic types, interfaces, or adapters;
- can become verbose if abstractions are poorly designed.

10.7 Static Versus Dynamic Typing

Strong/weak typing and static/dynamic typing are different dimensions.

Concept	Meaning
Static typing	Types of variables and expressions are fixed at compile time. Also called early binding in some contexts.
Dynamic typing	Types are known or checked at runtime. Also called late binding in some contexts.
Strong versus weak typing	Refers to how strictly type consistency is enforced.

10.8 Polymorphism

Polymorphism occurs when a single name or reference can denote objects of different classes related by a common superclass or interface, and those objects can respond to a common set of operations.

Listing 3: Polymorphism through a common interface.

```
interface Sensor {
    Measurement read();
}

final class TemperatureSensor implements Sensor {
    public Measurement read() {
        return new Measurement("temperature", 72.5);
    }
}
```

```

    }
}

final class HumiditySensor implements Sensor {
    public Measurement read() {
        return new Measurement("humidity", 54.0);
    }
}

final class SensorReporter {
    public void reportAll(List<Sensor> sensors) {
        for (Sensor sensor : sensors) {
            System.out.println(sensor.read());
        }
    }
}

```

The reporter does not need runtime type checks for every sensor. The shared protocol lets each object respond appropriately.

10.9 Runtime Type Identification

Runtime type identification can answer, “What kind of object is this right now?” It should be used carefully because it can weaken encapsulation and often signals that polymorphism is missing.

Design Smell

Large conditional chains such as `if sensor is TemperatureSensor` often indicate that behavior belongs inside the sensor classes through polymorphic operations.

11 Element 6: Concurrency

11.1 Definition

Concurrency is the property that distinguishes an active object from one that is not active.

An active object owns or represents an independent thread of control.

11.2 Why Concurrency Matters

Some systems must:

- respond to many events at once;
- monitor sensors continuously;
- handle user input while doing background work;
- distribute computation across processors;
- keep services responsive under load.

11.3 Thread of Control

Every program has at least one thread of control. Concurrent systems may have many:

- transient threads for short tasks;
- long-lived threads for services;
- active objects that run throughout system execution.

11.4 True Versus Simulated Concurrency

Form	Meaning
True concurrency	Multiple CPUs or cores execute threads at the same time.
Simulated concurrency	A single CPU gives the illusion of simultaneous execution through scheduling or time slicing.

11.5 Heavyweight Versus Lightweight Processes

Type	Characteristics	Communication Cost
Heavyweight process	Managed by the operating system; usually has its own address space.	Higher; uses interprocess communication.
Lightweight process / thread	Shares an address space with other threads in the same process.	Lower; may use shared memory but requires synchronization.

11.6 Concurrency Hazards

Hazard	Meaning
Race condition	Output depends on unpredictable timing of operations.
Deadlock	Two or more threads wait forever for resources held by each other.
Livelock	Threads keep reacting to each other but make no useful progress.
Starvation	A thread waits indefinitely because other work keeps taking priority.
Mutual exclusion failure	Shared state is updated concurrently without adequate protection.

11.7 Object Model View of Concurrency

The object model helps because objects can define:

- units of distribution;
- entities that communicate;
- boundaries where synchronization can be hidden;
- active objects that own independent behavior.

11.8 Hydroponics Example: Active Sensor

An `ActiveTemperatureSensor` may periodically read temperature and notify clients when values cross thresholds. Its public interface should not expose whether it uses:

1. built-in language concurrency;
2. a thread library;
3. hardware interrupts;
4. a scheduled executor;
5. an event loop.

Listing 4: Concurrency hidden inside an active object.

```
final class ActiveTemperatureSensor {
    private final TemperatureSensor sensor;
    private final ScheduledExecutorService scheduler;
    private volatile double setpoint;

    /*
    public void start() {
        scheduler.scheduleAtFixedRate(this::checkTemperature, 0, 1, TimeUnit.SECONDS);
    }

    public void establishSetpoint(double setpoint) {
        this.setpoint = setpoint;
    }

    private void checkTemperature() {
        double current = sensor.readTemperature();
        if (Math.abs(current - setpoint) > 2.0) {
            notifyClients(current);
        }
    }

    private void notifyClients(double current) {
        // Hidden notification policy.
    }
    */
}
```

11.9 Concurrency and Encapsulation

In sequential systems, an object must preserve invariants across method calls. In concurrent systems, it must preserve invariants even when multiple threads interact with it.

Critical Rule

When concurrency is introduced, method signatures are not enough. The design must specify synchronization semantics, atomicity expectations, and thread-safety guarantees.

12 Element 7: Persistence

12.1 Definition

Persistence is the property of an object through which its existence transcends time and/or space.

- **Persistence in time:** the object continues to exist after its creator or program execution ends.
- **Persistence in space:** the object can move across address spaces or machines.

12.2 Continuum of Object Existence

Lifetime Form	Example
Transient expression result	Temporary value created during calculation.
Local variable	Object created during a procedure or method activation.
Heap or global object	Object whose lifetime differs from local scope.
Between program executions	Saved user session, configuration, or domain object.
Between program versions	Data migrated across releases.
Outlives the program	Database records, durable business entities, archived objects.

12.3 Problem Persistence Solves

Many real systems must preserve state:

- user accounts;
- orders;
- medical records;
- sensor histories;
- growing plans;
- configuration;
- audit logs.

Without persistence, all object state disappears when the program ends.

12.4 Object-Relational Tension

Traditional programming languages handle transient objects well. Databases handle long-lived data well. The tension between the two leads to challenges:

- mapping classes to tables;
- preserving object identity;
- maintaining relationships;
- handling schema evolution;
- managing transactions;

- avoiding duplicated domain logic;
- resolving disconnected updates in distributed systems.

12.5 Approaches to Persistence

Approach	Benefits	Tradeoffs
Flat file streaming	Simple for small cases.	Naive; does not scale well for querying, concurrency, transactions, or schema evolution.
Object-oriented database	Preserves object concepts directly.	Less common; may be harder to integrate with existing relational infrastructure.
Object-relational mapping	Reuses relational databases while exposing object interfaces.	Mapping complexity, performance surprises, impedance mismatch.
Custom persistence layer	Full control.	Difficult to design correctly; high maintenance burden.

12.6 Persistence of Class and State

Persistence is not only about saving field values. A persistent object's class meaning must also remain interpretable across programs and versions. If a class changes, the stored objects may need migration.

Engineering Implication

Changing a persistent class is more dangerous than changing a purely transient class because old saved objects must still be interpreted correctly.

13 Applying the Object Model

13.1 Five Benefits of Applying the Object Model

1. **Exploits OO language power.** Languages such as Java, C++, C#, and Smalltalk are most powerful when their object-oriented mechanisms are used intentionally.
2. **Encourages reuse of software and designs.** Reusable classes, components, frameworks, and patterns become possible.
3. **Produces stable intermediate forms.** Systems become more resilient to requirements change.
4. **Reduces development risk.** Integration happens throughout the lifecycle rather than as one late “big bang.”
5. **Matches human cognition.** People naturally understand systems in terms of objects, categories, roles, and relationships.

13.2 How the Object Model Supports Complex-System Attributes

Complex-System tribute	At- Object Model Support
Hierarchy	Class and object hierarchies organize abstractions by generalization and composition.
Relative primitives	What counts as primitive depends on viewpoint: object, class, module, subsystem, service, or framework.
Separation of concerns	Abstraction, encapsulation, and modularity assign responsibilities and hide details.
Common patterns	Inheritance, aggregation, protocols, controllers, frameworks, and design patterns recur across systems.
Stable intermediate forms	Classes, modules, packages, and frameworks provide durable structures that can evolve.

13.3 Open Issues Introduced by the Chapter

The object model raises questions answered in later chapters:

- What exactly are classes and objects?
- How do we identify relevant classes and objects?
- What notation should describe object-oriented systems?
- What process leads to a well-structured object-oriented system?
- What management implications follow from object-oriented design?

14 UML and Architecture Implications

14.1 What UML Diagrams Communicate

Chapter 2 uses class-like diagrams to describe abstractions such as `TemperatureSensor`, `ActiveTemperatureSensor`, `GrowingPlan`, and `Heater`. These diagrams communicate:

- the name of an abstraction;
- important characteristics;
- responsibilities or operations;
- related candidate abstractions;
- inheritance relationships;
- aggregation or composition relationships.

14.2 When to Use UML for Chapter 2 Concepts

Diagram Type	Use It To Show	Common Mistake
Class diagram	Classes, attributes, operations, inheritance, associations.	Treating it as a database schema only.
Object diagram	Concrete object instances and links at a point in time.	Confusing objects with classes.
Sequence diagram	Message flow over time.	Showing implementation calls before responsibilities are understood.
State machine diagram	Object lifecycle and legal state transitions.	Ignoring invariants and invalid transitions.
Component/package diagram	Physical modularization and dependencies.	Using packages as arbitrary folders instead of design boundaries.

14.3 How to Interpret OO Diagrams

When reading a UML-style design:

1. Identify the abstraction names.
2. Ask whether each abstraction belongs to the problem domain, solution domain, or implementation domain.
3. Identify public responsibilities.
4. Identify hidden implementation decisions.
5. Distinguish ‘is-a’ from ‘part-of’ relationships.
6. Look for dependency direction.
7. Ask which changes are localized by the design.

14.4 Common Diagram Mistakes

- Drawing inheritance when aggregation is intended.
- Modeling every field as public data.
- Creating diagrams that mirror code syntax but not design intent.
- Omitting dynamic behavior.
- Ignoring module/package boundaries.
- Confusing responsibilities with implementation steps.

15 Design Tradeoffs Across Object Model Elements

Element	Benefit	Tradeoff	Risk if Misused
Abstraction	Reduces conceptual complexity.	May hide details needed for performance or correctness decisions.	Vague objects with unclear responsibilities.
Encapsulation	Localizes change and protects invariants.	Can add indirection and require careful interface design.	Getter/setter objects with leaked representation.
Modularity	Supports team development and physical organization.	Too many modules create overhead.	Arbitrary packaging and dependency tangles.
Hierarchy	Organizes abstractions and enables reuse.	Inheritance can weaken encapsulation.	False “is-a” relationships and brittle base classes.
Typing	Enforces design contracts.	Can reduce flexibility or increase recompilation costs.	Overly rigid or unsafe casts.
Concurrency	Models independent activity and responsiveness.	Introduces synchronization complexity.	Race conditions, deadlock, corrupted state.
Persistence	Preserves object state across time/space.	Requires mapping, migration, and consistency mechanisms.	Fragile storage formats and schema drift.

16 Key Takeaways

1. The object model is a synergistic combination of abstraction, encapsulation, modularity, hierarchy, typing, concurrency, and persistence.
2. Object-oriented development is evolutionary, not revolutionary; it builds on structured programming, abstract data types, information hiding, databases, AI, and cognitive science.
3. OOP requires objects, classes, and inheritance. Without inheritance, the result is closer to abstract data type programming.
4. OOD differs from structured design because it decomposes systems around classes and objects

rather than algorithms.

5. OOA models requirements using classes and objects from the problem-domain vocabulary.
6. Abstraction defines what an object is and does; encapsulation hides how it does it.
7. Modularity packages logical abstractions into physical units that can be developed, compiled, reused, documented, and managed.
8. Hierarchy organizes abstractions using ‘is-a’ inheritance and ‘part-of’ aggregation/composition.
9. Typing uses the programming language to enforce design constraints.
10. Concurrency introduces active objects and independent threads of control but requires synchronization discipline.
11. Persistence allows objects to outlive the program or move across address spaces.
12. The object model produces systems that better support reuse, evolution, integration, and human understanding.

17 Common Pitfalls

17.1 Pitfall 1: Treating Classes as Records

A class should not merely expose data fields. It should define responsibilities, preserve invariants, and provide behavior.

17.2 Pitfall 2: Confusing Abstraction With Vagueness

A good abstraction is not vague. It has crisp conceptual boundaries and a clear contract.

17.3 Pitfall 3: Exposing Too Much Interface

A broad interface creates unnecessary dependencies. Expose only the operations clients truly need.

17.4 Pitfall 4: Using Inheritance for Code Reuse Alone

Inheritance should express a valid “is-a” relationship. If the relationship is not semantic, prefer composition.

17.5 Pitfall 5: Creating Arbitrary Modules

Modules should group cohesive abstractions and hide meaningful secrets. A file-per-class strategy is not automatically modular design.

17.6 Pitfall 6: Ignoring Object Lifetimes

Persistent objects, composed objects, aggregated objects, and transient objects have different lifetime rules. Confusing them leads to memory, data consistency, and ownership errors.

17.7 Pitfall 7: Adding Concurrency Without Redesigning Semantics

Thread-safe behavior is not achieved by sprinkling locks randomly. The object’s invariants, atomic operations, and synchronization policy must be designed.

17.8 Pitfall 8: Overusing Runtime Type Checks

Frequent type checks often mean polymorphism is missing or responsibilities are assigned to the wrong object.

18 Interview Questions

18.1 Question 1

Question: What is the object model?

Concise Answer: The object model is the conceptual foundation of object-oriented development. It combines abstraction, encapsulation, modularity, hierarchy, typing, concurrency, and persistence to organize complex software around objects, classes, responsibilities, and collaborations.

18.2 Question 2

Question: How is object-oriented decomposition different from algorithmic decomposition?

Concise Answer: Algorithmic decomposition organizes a system around procedures and control flow. Object-oriented decomposition organizes a system around objects that combine state and behavior. OO decomposition is often more stable for complex domains because real-world entities and responsibilities tend to change less frequently than procedural steps.

18.3 Question 3

Question: What three features are required for a program to be considered object-oriented in the strict sense?

Concise Answer: The program must use objects as fundamental logical building blocks, each object must be an instance of a class, and classes must participate in inheritance relationships. Without inheritance, the design is closer to abstract data type programming.

18.4 Question 4

Question: What is the difference between object-oriented analysis, design, and programming?

Concise Answer: Object-oriented analysis identifies problem-domain classes and objects from requirements. Object-oriented design structures those objects into logical and physical models. Object-oriented programming implements the design using language mechanisms such as classes, objects, inheritance, and polymorphism.

18.5 Question 5

Question: What problem does abstraction solve?

Concise Answer: Abstraction reduces complexity by emphasizing essential characteristics and suppressing irrelevant implementation details. It lets engineers reason about what an object is responsible for before deciding how it is implemented.

18.6 Question 6

Question: What is the difference between abstraction and encapsulation?

Concise Answer: Abstraction defines the outside view of an object: its essential behavior and responsibilities. Encapsulation protects the inside view by hiding representation and implementation details behind a stable interface.

18.7 Question 7

Question: Why is information hiding important?

Concise Answer: Information hiding localizes design decisions that are likely to change. If clients depend only on a stable interface, the implementation can change without breaking the rest of the system.

18.8 Question 8

Question: What makes a module cohesive and loosely coupled?

Concise Answer: A cohesive module groups abstractions that belong together conceptually or change together. A loosely coupled module exposes a narrow interface and minimizes assumptions other modules must make about its internals.

18.9 Question 9

Question: When should inheritance be used?

Concise Answer: Inheritance should be used when a subclass is genuinely a specialized kind of its superclass. The relationship must pass the “is-a” test. If the goal is only reuse of implementation, composition is often safer.

18.10 Question 10

Question: What is the difference between aggregation and composition?

Concise Answer: Aggregation is a weak whole-part relationship where parts can exist independently of the whole. Composition is a strong ownership relationship where parts are intrinsic to the whole and usually share its lifetime.

18.11 Question 11

Question: What is polymorphism, and why is it important?

Concise Answer: Polymorphism allows one reference or operation name to work with objects of different classes that share a common superclass or interface. It reduces conditional logic and supports extensibility because new subclasses can be added without changing client code.

18.12 Question 12

Question: What is the difference between strong typing and static typing?

Concise Answer: Strong typing concerns how strictly type consistency is enforced. Static typing concerns when types are known: compile time rather than runtime. A language can be strongly typed and still support dynamic binding.

18.13 Question 13

Question: Why is concurrency difficult in object-oriented systems?

Concise Answer: Concurrency introduces multiple threads of control that may interact with the same objects. Designers must preserve invariants under simultaneous access, avoiding race conditions, deadlock, livelock, starvation, and inconsistent state.

18.14 Question 14

Question: What is an active object?

Concise Answer: An active object owns or represents an independent thread of control. Unlike a passive object that only acts when called, an active object can initiate behavior concurrently with other objects.

18.15 Question 15

Question: What is persistence in the object model?

Concise Answer: Persistence is the property that allows an object to outlive the process that created it or move across address spaces. Persistent systems must preserve both object state and the meaning of the object's class across time or space.

18.16 Question 16

Question: Why does the object model improve reuse?

Concise Answer: The object model packages stable abstractions behind clear interfaces and organizes related abstractions through inheritance, aggregation, and modules. This supports reuse of individual classes, frameworks, and complete design structures.

18.17 Question 17

Question: Why can object-oriented systems be more resilient to change?

Concise Answer: They are built from stable intermediate forms: classes, modules, interfaces, and frameworks. When responsibilities and secrets are well assigned, changes tend to be localized rather than spread across the entire system.

18.18 Question 18

Question: What is a common mistake candidates make when explaining encapsulation?

Concise Answer: A common mistake is saying encapsulation only means making fields private. Strong encapsulation means hiding all implementation decisions that clients should not depend on, including data structures, algorithms, protocols, caching, and synchronization.

19 Engineering Insights

19.1 Insight 1: Object-Oriented Design Is Responsibility Assignment

The central question is not ‘What classes can I create?’ but ‘Which object should be responsible for which behavior?’ Good responsibility assignment leads to cohesive abstractions and simpler collaboration.

19.2 Insight 2: Interfaces Are Commitments

Every public method becomes a promise to clients. The broader the interface, the harder the system is to change.

19.3 Insight 3: Reuse Requires Upfront Design Investment

Reusable components usually cost more to design initially because they need stable interfaces, documentation, tests, and fewer hidden assumptions. The payoff comes in later reuse.

19.4 Insight 4: Inheritance Is a Semantic Tool, Not Just a Coding Tool

Inheritance communicates domain meaning. Misusing it for convenience creates brittle designs.

19.5 Insight 5: Persistence and Concurrency Change Object Semantics

A class that is correct in memory and single-threaded may become incorrect when objects are saved, restored, shared, distributed, or accessed concurrently.

19.6 Insight 6: Modules Are Management Boundaries

Modules are not just compiler artifacts. They influence team ownership, documentation, release planning, security, and communication.

20 Connections to Other Topics

20.1 Connection to Chapter 1: Complexity

Chapter 1 explained that complex systems are hierarchical, nearly decomposable, and built from stable intermediate forms. Chapter 2 shows how the object model realizes those ideas in software through classes, objects, modules, inheritance, aggregation, and frameworks.

20.2 Connection to SOLID Principles

SOLID Principle		Connection to Chapter 2
Single Responsibility Principle		Refines abstraction and cohesion by ensuring each class has one focused reason to change.
Open/Closed Principle		Uses abstraction, inheritance, and polymorphism to extend behavior without modifying stable code.
Liskov Substitution Principle		Formalizes correct “is-a” relationships and safe subtype substitution.
Interface Segregation Principle		Encourages narrow interfaces and avoids forcing clients to depend on operations they do not use.
Dependency Inversion Principle		Depends on stable abstractions rather than volatile implementation details.

20.3 Connection to Design Patterns

Design patterns rely heavily on Chapter 2 concepts:

- Strategy uses abstraction and polymorphism to replace conditional behavior.
- Observer models active collaboration and notification.
- Factory Method and Abstract Factory encapsulate object creation.
- Adapter protects clients from incompatible interfaces.
- Composite models part-whole hierarchies.
- Proxy controls access and can support persistence, distribution, or lazy loading.

20.4 Connection to Software Architecture

Architecture applies the same principles at a larger scale:

- services are higher-level modules;
- APIs are system-level interfaces;
- microservices require strong boundaries and explicit contracts;
- persistence affects data ownership and schema evolution;
- concurrency affects scalability and correctness;
- frameworks are reusable designs embodied in code.

20.5 Connection to Testing

The object model improves testability when:

- objects have clear responsibilities;
- interfaces are narrow;
- dependencies are explicit;
- invariants can be tested;
- polymorphic collaborators can be replaced with test doubles.

21 Practical Applications

21.1 Application 1: Designing a Domain Model

Use abstraction to identify domain concepts such as `Order`, `Customer`, `Invoice`, or `GrowingPlan`. Assign responsibilities to the objects that naturally own the information and behavior.

21.2 Application 2: Refactoring Procedural Code

Procedural code with long functions and global data can often be improved by:

1. identifying the data that changes together;
2. naming the domain concept represented by that data;
3. moving related behavior into the object;
4. hiding representation behind methods;

5. introducing polymorphism where conditional behavior varies by type.

21.3 Application 3: Designing Package Boundaries

Group classes by cohesive domain capability rather than by technical layer alone. For example, `growingplan`, `sensors`, and `control` may be more meaningful than dumping all classes into `models`, `utils`, and `services`.

21.4 Application 4: Choosing Inheritance or Composition

Use inheritance when the subclass is substitutable for the superclass. Use composition when an object has or uses another object.

Listing 5: Prefer composition when behavior is a role rather than an identity.

```
// Better: a Greenhouse has sensors.
final class Greenhouse {
    private final List<Sensor> sensors;
}

// Risky: a Greenhouse is not a Sensor.
final class Greenhouse extends Sensor {
    // This would fail the is-a test.
}
```

21.5 Application 5: Designing Persistent Domain Objects

When objects must persist:

- define stable identities;
- separate persistence concerns from domain behavior where possible;
- plan for schema evolution;
- avoid exposing database details in domain interfaces;
- design transactions around invariants.

21.6 Application 6: Designing Concurrent Objects

When objects are accessed concurrently:

- identify shared mutable state;
- define atomic operations;
- document thread-safety guarantees;
- encapsulate locks or synchronization;
- avoid callbacks while holding locks when possible;
- test race-prone behavior under load.

22 Summary Cheat Sheet

22.1 Object Model Elements

Element	One-Sentence Definition	Exam Keyword
Abstraction	Essential characteristics that distinguish an object and define its conceptual boundary.	Outside view.
Encapsulation	Separation of an abstraction's interface from its implementation.	Information hiding.
Modularity	Decomposition into cohesive, loosely coupled physical units.	Packages/modules.
Hierarchy	Ranking or ordering of abstractions.	Is-a and part-of.
Typing	Enforcement of object class/type compatibility.	Type safety.
Concurrency	Distinguishes active objects with independent control from passive objects.	Threads/processes.
Persistence	Allows object existence to transcend time or space.	Durable objects.

22.2 Most Important Comparisons

Concept A	Concept B	Difference
OOA	OOD	OOA models requirements; OOD structures a solution.
OOD	OOP	OOD creates design models; OOP implements them.
Abstraction	Encapsulation	Abstraction defines what is visible; encapsulation hides how it works.
Class hierarchy	Object hierarchy	Class hierarchy is 'is-a'; object hierarchy is 'part-of.'
Inheritance	Aggregation	Inheritance means specialization; aggregation means containment or membership.
Strong typing	Static typing	Strong typing is strictness; static typing is compile-time binding.

Concept A	Concept B	Difference
Passive object	Active object	Passive objects respond when called; active objects initiate behavior through independent control.
Transient object	Persistent object	Transient objects die with execution context; persistent objects survive beyond it.

22.3 Fast Exam Review

- **Object model** = abstraction + encapsulation + modularity + hierarchy + typing + concurrency + persistence.
- **OOP** = objects + classes + inheritance.
- **OOD** = object-oriented decomposition + notation for logical/physical and static/dynamic models.
- **OOA** = requirements analysis through problem-domain classes and objects.
- **Best abstractions** are usually entity abstractions from the domain vocabulary.
- **Encapsulation** hides secrets likely to change.
- **Modularity** is physical packaging; class/object identification is logical design.
- **Inheritance** must pass the ‘is-a’ test.
- **Aggregation/composition** model ‘part-of’ relationships and ownership.
- **Polymorphism** reduces conditionals and enables extension.
- **Concurrency** requires preserving object invariants across multiple threads.
- **Persistence** requires preserving both state and class meaning across time or space.

22.4 One-Page Mental Model

